



TẬP ĐOÀN BƯU CHÍNH VIỄN THÔNG VIỆT NAM
VNPT IT - TRUNG TÂM DMS
Bộ phận Phát triển Giải pháp AI

BÁO CÁO KIỂM THỬ HIỆU NĂNG HỆ THỐNG HỎI ĐÁP DỮ LIỆU TEXT-TO-SQL DƯỚI TẢI NGƯỜI DÙNG ĐỒNG THỜI

Load Test bằng công cụ k6 — Xác định ngưỡng bão hoà
trên nền tảng Vanna, Ollama 1lama3.2 và PostgreSQL Neon

PHẠM VI KIỂM THỬ

Báo cáo trình bày kết quả kiểm thử tải (load test) endpoint hỏi đáp AI /api/vanna/v2/chat_sse của hệ thống PoC Text-to-SQL. Mục tiêu là mô phỏng số lượng người dùng đồng thời tăng dần (5, 15, 30 và 50 người dùng), đo thời gian phản hồi, thông lượng và tỉ lệ lỗi, từ đó xác định điểm bão hoà và đề xuất phương án nâng cao khả năng chịu tải. Báo cáo cũng ghi nhận **đợt tối ưu tầng ứng dụng** được thực hiện sau kiểm thử lần đầu và kết quả đo lại ở cùng mức tải 50 người dùng.

Tóm tắt thông số kiểm thử

Công cụ kiểm thử:	k6 v2.1.0, kịch bản <code>ramping-vus</code>
Đối tượng:	Endpoint SSE của Vanna 2.0 (FastAPI, Uvicorn 1 worker)
Hạ tầng AI:	Ollama 1lama3.2 cục bộ, PostgreSQL trên Neon
Mức tải:	5 → 15 → 30 người dùng và cao điểm 50 người dùng
Kết quả trước tối ưu:	50 user: tỉ lệ lỗi 13%, p95 = 139 giây
Kết quả sau tối ưu:	50 user: tỉ lệ lỗi 0%, p95 = 46,3 giây, thông lượng +766%
Nút thắt còn lại:	Tốc độ sinh token của LLM trên CPU (cần gấp ≈ 5 lần)

Người thực hiện:
Lê Đức Anh
Trung tâm DMS

Mục lục

1	Tóm tắt kết quả cho Lãnh đạo (Executive Summary)	2
2	Môi trường và phương pháp kiểm thử	2
3	Kết quả kiểm thử	5
4	Phân tích kết quả	7
5	Định chính phương pháp đo	8
6	Tối ưu hoá tầng ứng dụng và kết quả đo lại	9
7	Thử nghiệm cấu hình tầng suy luận	12
8	Đánh giá theo tiêu chí nghiệm thu (SLA)	15
9	Giới hạn của kiểm thử	16
10	Kết luận	17
11	Khuyến nghị cải thiện	17
	Phụ lục A. Kịch bản k6	20
	Tài liệu tham khảo	22

1. Tóm tắt kết quả cho Lãnh đạo (Executive Summary)

Báo cáo đánh giá khả năng chịu tải của hệ thống hỏi đáp dữ liệu bằng ngôn ngữ tự nhiên (Text-to-SQL) khi số lượng người dùng truy cập đồng thời tăng cao. Công cụ **k6** được dùng để phát sinh tải thực tế lên endpoint streaming `/api/vanna/v2/chat_sse`, mô phỏng các câu hỏi nghiệp vụ tiếng Việt.

Kiểm thử được thực hiện qua **hai đợt**. Đợt đầu đo hiện trạng: hệ thống vận hành ổn định dưới 10 người dùng đồng thời, nhưng khi vượt ngưỡng này thì bão hoà — thông lượng không tăng thêm mà độ trễ và tỉ lệ lỗi tăng phi tuyến. Ở mức 50 người dùng, độ trễ p95 đạt **139 giây** và tỉ lệ lỗi lên tới **13%**.

Phân tích mã nguồn sau đợt đầu phát hiện nguyên nhân **không nằm ở câu lệnh SQL hay ở cơ sở dữ liệu**, mà ở cách tầng ứng dụng gọi chúng: thư viện truy cập PostgreSQL và thư viện gọi Ollama đều là thư viện **đồng bộ** nhưng được gọi trực tiếp trong vòng lặp sự kiện bất đồng bộ (asyncio) của máy chủ. Hệ quả là mỗi lần chờ cơ sở dữ liệu hoặc chờ mô hình sinh chữ, **toàn bộ tiến trình bị đóng băng** — 50 người dùng đồng thời trên thực tế bị phục vụ nối đuôi nhau. Ngoài ra mỗi truy vấn còn mở một kết nối mới tới Neon, tốn một lượt bắt tay TLS xuyên quốc gia trước khi chạy được câu lệnh.

Sau khi khắc phục (Mục 6) và phát hiện thêm rằng `OLLAMA_NUM_PARALLEL` để mặc định bằng 1 — tức máy chủ mô hình xử lý tuần tự — nâng tham số này lên 4 (Mục 7), kiểm thử lại ở đúng mức 50 người dùng cho kết quả: tỉ lệ thành công **100%** (từ 87%), thông lượng **tăng 766%** lên 6,06 req/giây và p95 **giảm 67%** xuống 46,3 giây.

Lưu ý về tính hiệu lực của số liệu. Bản báo cáo phát hành trước đó chứa các con số “sau tối ưu” thu thập bằng một quy trình có ba sai sót nghiêm trọng — trong đó có việc **toàn bộ truy vấn cơ sở dữ liệu đã hỏng trong suốt đợt đo mà bộ kiểm thử không phát hiện được**. Toàn bộ số liệu trong bản này là kết quả **đo lại** sau khi khắc phục cả ba. Chi tiết ở Mục 5; đây cũng là bài học đáng lưu ý nhất của đợt kiểm thử.

Kết luận nhanh

Đợt tối ưu tầng ứng dụng đã **loại bỏ hoàn toàn lỗi** ở mức 50 người dùng và nâng thông lượng từ 0,70 lên 6,06 request/giây. Độ trễ tách làm hai: năm trong sáu câu hỏi được trả lời trong **dưới nửa giây** ở mọi mức tải, còn câu cần mô hình suy luận vẫn mất hàng chục giây khi đông người dùng. Nút thắt còn lại vì vậy nằm gọn ở **tốc độ suy luận trên CPU**: khi máy rảnh một câu như vậy mất 1,99 giây, dưới 50 người dùng đồng thời lên 45 giây — chênh lệch là thời gian xếp hàng. Đây là giới hạn phần cứng, không gỡ tiếp được bằng mã nguồn; muốn xuống dưới 10 giây cần GPU rời hoặc API LLM đám mây [3, 2].

2. Môi trường và phương pháp kiểm thử

2.1. Cấu hình môi trường

Bảng 1: Cấu hình môi trường kiểm thử

Hạng mục	Chi tiết
Công cụ kiểm thử	k6 v2.1.0 (Windows/amd64)
Endpoint	POST /api/vanna/v2/chat_sse (Server-Sent Events)
Mô hình AI	Ollama llama3.2 (chạy cục bộ, không tốn phí API)
Cơ sở dữ liệu	PostgreSQL — Neon, schema qlsp_backup
Máy chủ ứng dụng	Uvicorn (1 worker), FastAPI, Vanna 2.0
Loại truy vấn	Câu hỏi tiếng Việt (đếm và tổng hợp đơn giản)
Tiêu chí thành công	HTTP 200, stream kết thúc bằng [DONE], và thân phản hồi chứa sự kiện dataframe (có dữ liệu thật)
Timeout request	mỗi 180 giây
Baseline (1 người dùng)	≈ 2,5 giây/truy vấn (trước tối ưu)
Baseline sau tối ưu	0,105 giây (câu chỉ chạm CSDL) — 1,99 giây (câu gọi LLM)

2.2. Cấu hình phần cứng

Toàn bộ hệ thống (ứng dụng, mô hình Ollama và công cụ k6) chạy trên cùng một máy trạm; cơ sở dữ liệu PostgreSQL đặt từ xa trên Neon (cloud). Đây là mô hình kiểm thử một máy (single-node), phù hợp cho giai đoạn PoC.

Bảng 2: Thông số phần cứng máy chạy kiểm thử

Thành phần	Thông số
CPU	AMD Ryzen 7 8845H (8 nhân / 16 luồng)
GPU	AMD Radeon 780M (đồ hoạ tích hợp)
RAM	16 GB (khoảng 13,8 GB khả dụng; phần còn lại chia sẻ cho GPU tích hợp)
Hệ điều hành	Windows 11 Home
Vị trí cơ sở dữ liệu	Neon (PostgreSQL đám mây, kết nối qua SSL)

Ghi chú: Do máy sử dụng **GPU tích hợp** (không có GPU rời), mô hình llama3.2 suy luận hoàn toàn trên CPU. Lệnh `ollama ps` trong lúc kiểm thử xác nhận mô hình chạy ở chế độ 100% CPU. Đây là yếu tố phần cứng trực tiếp giới hạn tốc độ và mức độ song song của mô hình, và cần được tính đến khi diễn giải kết quả kiểm thử.

2.3. Đặc điểm kỹ thuật ảnh hưởng đến hiệu năng

- Mỗi request sử dụng một `conversation_id` riêng biệt để tránh cơ chế khoá theo hội thoại (HTTP 409) của máy chủ, bảo đảm tải thực sự song song.

- Trong 6 câu hỏi kiểm thử, **5 câu đi theo nhánh sinh SQL tắt định** của `main.py` (hệ thống nhận dạng mẫu câu đếm và dựng thẳng câu lệnh SQL, **không gọi mô hình**). Chỉ câu “Liệt kê 5 đơn vị đầu tiên theo tên” thực sự gọi `llama3.2`. Vì vậy chi phí LLM tập trung vào $\approx 1/6$ lượng request, nhưng đủ để chiếm trọn CPU và kéo chậm toàn bộ phần còn lại.
- Ollama cục bộ suy luận trên CPU nên là điểm giới hạn thông lượng chính của hệ thống. Đo trực tiếp qua API `/api/chat`: nạp prompt hệ thống (1.202 token) chỉ mất 0,28 giây, nhưng **sinh 118 token mất 6,53 giây**, tương đương ≈ 18 token/giây.
- Máy chủ FastAPI chạy bất đồng bộ (asyncio) nhưng **trước tối ưu** lại gọi thư viện đồng bộ (`psycopg2`, `ollama`) ngay trong vòng lặp sự kiện, khiến các request không thực sự chạy song song. Chi tiết ở Mục 4.

2.4. Kịch bản kiểm thử

Hai kịch bản tăng tải (`ramping-vus`) được thực hiện:

1. **Kịch bản A — Tăng dần:** 5 $\rightarrow$ 15 $\rightarrow$ 30 người dùng đồng thời, tổng thời gian $\approx$ 3 phút 35 giây. Mục tiêu quan sát diễn biến khi tải tăng từ thấp đến trung bình.
2. **Kịch bản B — Cao điểm 50:** tăng nhanh lên 50 người dùng và giữ trong 120 giây. Mục tiêu kiểm tra hành vi khi vượt xa ngưỡng bảo hoà.

Listing 1: Trích cấu hình kịch bản tăng tải trong `k6`

```
export const options = {
  scenarios: {
    ramping_users: {
      executor: 'ramping-vus',
      stages: [
        { duration: '20s', target: 5 }, // 5 người dùng
        { duration: '20s', target: 15 }, // tăng lên 15
        { duration: '60s', target: 30 }, // cao điểm 30
      ],
    },
  },
};
```

2.5. Bộ câu hỏi kiểm thử

Mỗi người dùng ảo chọn ngẫu nhiên một câu hỏi trong tập câu hỏi nghiệp vụ tiếng Việt dưới đây. Các câu hỏi thuộc nhóm đếm và tổng hợp đơn giản nhằm cô lập chi phí suy luận của mô hình (tách khỏi chi phí truy vấn SQL phức tạp):

1. “Có bao nhiêu người dùng trong hệ thống?”
2. “Có bao nhiêu đơn vị trong bảng `units`?”
3. “Có bao nhiêu `spdv_tickets`?”
4. “Đếm số người dùng theo trạng thái.”
5. “Liệt kê 5 đơn vị đầu tiên theo tên.”

6. “Có bao nhiêu sự kiện trong `spdv_ticket_events`?”

3. Kết quả kiểm thử

Mục này trình bày kết quả **đợt kiểm thử thứ nhất** (đo hiện trạng). Kết quả đo lại sau khi tối ưu được trình bày ở Mục 6.

3.1. Kết quả tổng quan

Bảng 3: So sánh kết quả tổng quan theo mức tải

Chỉ số	1 user (baseline)	30 user (A)	50 user (B)
Tổng số request	1	177	132
Thông lượng (req/giây)	0,40	0,75	0,70
Tỉ lệ thành công	100%	95,0%	87,0%
Số lỗi (rớt kết nối EOF)	0	9	17
Tỉ lệ lỗi HTTP	0%	5,0%	13,0%

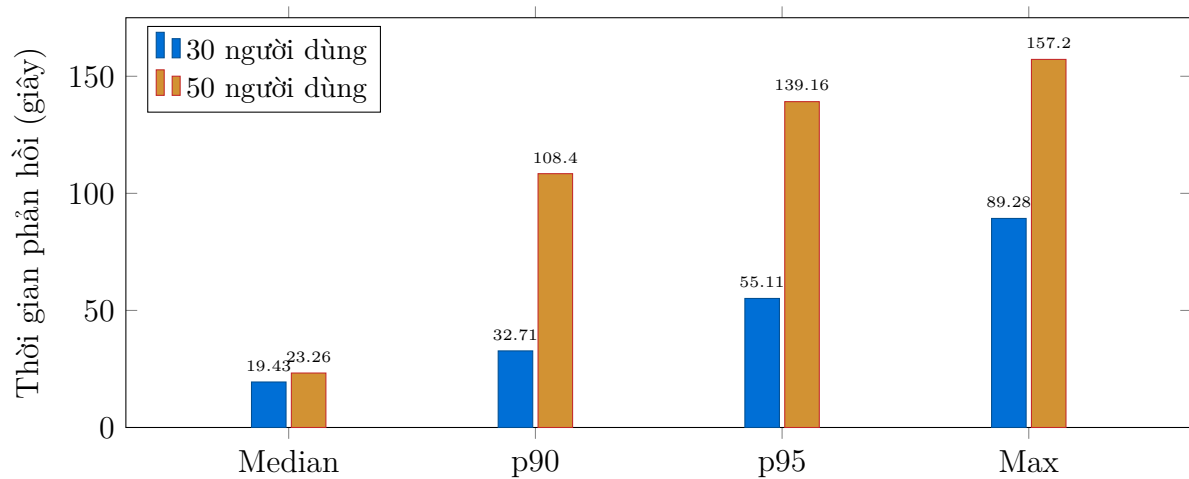
3.2. Phân bố thời gian phản hồi AI

Bảng 4: Phân bố thời gian phản hồi AI theo mức tải (đơn vị: giây)

Thời gian phản hồi	Baseline	30 user	50 user
Nhanh nhất (min)	–	0,59	1,34
Trung vị (median)	2,5	19,43	23,26
Trung bình (avg)	2,5	20,90	36,28
p90	–	32,71	108,40
p95	–	55,11	139,16
Chậm nhất (max)	2,5	89,28	157,20

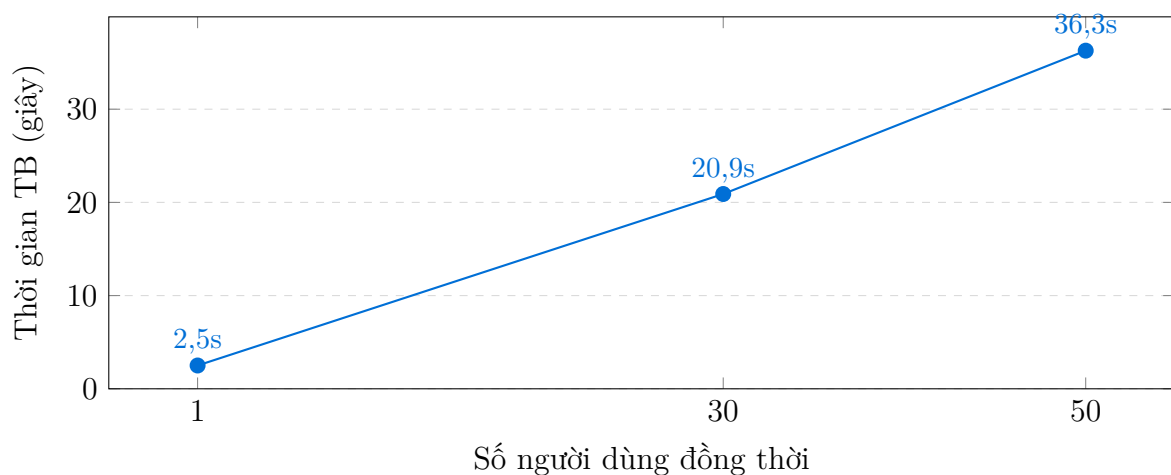
3.3. Biểu đồ trực quan

Hình 1 so sánh phân bố thời gian phản hồi giữa hai mức tải. Có thể thấy độ trễ tăng rất mạnh ở các percentile cao khi chuyển từ 30 lên 50 người dùng: p95 nhảy từ 55 giây lên 139 giây.



Hình 1: So sánh phân bố thời gian phản hồi AI ở mức 30 và 50 người dùng đồng thời.

Hình 2 cho thấy đặc trưng bão hoà: thời gian phản hồi trung bình tăng gần tuyến tính theo số người dùng, trong khi thông lượng không tăng — dấu hiệu điển hình của một hệ thống đã chạm trần xử lý.



Hình 2: Thời gian phản hồi trung bình tăng gần tuyến tính theo số người dùng (baseline → 30 → 50), trong khi thông lượng giữ nguyên $\approx 0,7$ req/giây.

3.4. Diễn biến theo mức tải

Bảng 5: Hành vi hệ thống quan sát theo từng giai đoạn tải

Giai đoạn	Số user	Hành vi hệ thống
Tải thấp	5	Ổn định, phản hồi nhanh, không có lỗi
Tải trung	15	Độ trễ tăng rõ, xuất hiện lỗi rút kết nối đầu tiên
Tải cao	30	Thông lượng chững lại, nhiều lỗi EOF, tỉ lệ lỗi 5%
Quá tải	50	Bão hoà hoàn toàn, p95 tới 139 giây, tỉ lệ lỗi 13%

4. Phân tích kết quả

4.1. Thông lượng đạt trần

Thông lượng gần như không đổi giữa 30 và 50 người dùng (0,75 so với 0,70 request/giây). Điều này chứng minh hệ thống đã **đạt trần xử lý** ở mức $\approx 0,7-0,75$ request/giây. Việc tăng thêm người dùng không làm tăng năng lực xử lý mà chỉ kéo dài hàng đợi, khiến độ trễ và tỉ lệ lỗi tăng vọt.

4.2. Giải thích bằng định luật Little

Định luật Little trong lý thuyết hàng đợi phát biểu:

$$L = \lambda \times W$$

trong đó L là số yêu cầu trung bình đang trong hệ thống (mức đồng thời), λ là thông lượng (request/giây) và W là thời gian phản hồi trung bình. Khi hệ thống đã bão hoà, thông lượng λ bị chặn trên bởi năng lực xử lý:

$$\lambda_{\max} \approx \frac{N_{\text{song song}}}{W_{\text{phục vụ}}}$$

Áp dụng cho đợt đo thứ nhất ở mức 50 người dùng: $\lambda = 0,70$ req/giây và $W = 36,28$ giây cho $L \approx 25$ — tức trung bình chỉ khoảng 25 trong số 50 người dùng thực sự có yêu cầu đang được hệ thống giữ, phần còn lại đã bị huỷ do lỗi. Với thời gian phục vụ cơ sở $W_{\text{phục vụ}} \approx 2,5$ giây đo ở baseline, mức song song hiệu dụng chỉ đạt $N_{\text{song song}} \approx 0,70 \times 2,5 \approx 1,8$.

Con số $\approx 1,8$ này ban đầu được quy cho giới hạn song song của Ollama. Tuy nhiên rà soát mã nguồn (Mục 6) cho thấy nó phản ánh một **điểm tuần tự hoá ở tầng ứng dụng**: máy chủ chỉ có một vòng lặp sự kiện và vòng lặp này bị các lời gọi đồng bộ chiếm dụng, nên dù có bao nhiêu người dùng thì mức song song hiệu dụng vẫn xấp xỉ 1. Đây là lý do sau khi khắc phục, thông lượng tăng lên 4,43 req/giây — và lên 6,06 req/giây sau khi chỉnh thêm NUM_PARALLEL — mà **không cần thay đổi gì về phần cứng**.

4.3. Hai nút thắt tách biệt

Đợt phân tích sau kiểm thử xác định **hai** nút thắt độc lập, trước đây bị gộp làm một:

1. **Tuần tự hoá ở tầng ứng dụng (đã khắc phục)**. Thư viện `psycpg2` và thư viện `ollama` đều đồng bộ nhưng được gọi trực tiếp trong hàm `async`. Mỗi lần chờ mạng tới Neon hoặc chờ mô hình sinh chữ, cả tiến trình dừng lại, không phục vụ được người dùng nào khác. Thêm vào đó, mỗi truy vấn mở một kết nối PostgreSQL mới, tốn trọn một lượt bắt tay TLS tới Neon (Singapore) trước khi câu lệnh được chạy.
2. **Tốc độ suy luận trên CPU (chưa khắc phục được bằng mã nguồn)**. Đo trực tiếp: ≈ 18 token/giây. Khoảng 1/6 số request cần gọi mô hình, mỗi lần tốn $\approx 5,5$ giây CPU. Trong 190 giây kiểm thử có ≈ 34 lượt gọi như vậy, tổng cộng ≈ 187 giây CPU — **bằng đúng thời lượng bài kiểm thử**. Nói cách khác CPU bị chiếm dụng 100% bởi suy luận, và chính điều này kéo chậm cả những request vốn chỉ chạm cơ sở dữ liệu.

4.4. Lỗi rớt kết nối

Toàn bộ lỗi ở đợt đo thứ nhất thuộc dạng EOF, tỉ lệ tăng từ 5% (30 người dùng) lên 13% (50 người dùng). Nguyên nhân trực tiếp là nút thắt số 1 ở trên: khi vòng lặp sự kiện bị chiếm dụng, request xếp hàng vượt quá timeout 180 giây và bị hủy. Sau khi khắc phục, **tỉ lệ lỗi về 0%** ở cùng mức tải — xác nhận lỗi đến từ tầng ứng dụng chứ không phải từ giới hạn kết nối của Neon.

Nhận định trọng yếu

Kết luận ban đầu “nút thắt duy nhất là mô hình ngôn ngữ” là **chưa đầy đủ**. Phần lớn lỗi và phần lớn độ trễ đuôi ở mức 50 người dùng đến từ cách tầng ứng dụng gọi các thư viện đồng bộ — sửa được bằng mã nguồn, chi phí bằng không. Sau khi gỡ lớp này, nút thắt thật sự mới lộ ra là tốc độ sinh token trên CPU, và đây là bài toán phần cứng.

5. Đỉnh chính phương pháp đo

Bản báo cáo phát hành ngày 05/08/2026 chứa số liệu “sau tối ưu” được thu thập bằng một quy trình có ba sai sót. Cả ba đều làm sai lệch kết quả theo hướng **không sửa được bằng cách diễn giải lại** — bắt buộc phải đo lại. Mục này ghi rõ từng sai sót và cách phát hiện; toàn bộ số liệu ở Mục 6 và Mục 7 sau đây là kết quả **đo lại** sau khi khắc phục.

5.1. Sai sót 1: toàn bộ truy vấn cơ sở dữ liệu đã hỏng

Bản vá tối ưu đường truy vấn đưa vào hàm `_prepare_connection` một cờ đánh dấu để mỗi kết nối chỉ phải cấu hình một lần:

```
def _prepare_connection(self, conn) -> None:
    if getattr(conn, "_vanna_prepared", False):
        return
    conn.autocommit = True
    ...
    conn._vanna_prepared = True # <-- AttributeError tại đây
```

Kiểu `connection` của `psycopg2` viết bằng C và không có `__dict__`, nên phép gán thuộc tính tùy ý ở dòng cuối ném `AttributeError`. Lời gọi `_prepare_connection` lại nằm ngay đầu khối `try` của hàm thực thi, vì vậy **mọi lời gọi `run_sql` đều ném lỗi trước khi chạm tới câu lệnh SQL**, kể từ bản vá tối ưu cho tới khi được phát hiện.

Sai sót này không để lại dấu vết trong nhật ký. Tầng ứng dụng bất ngoại lệ của công cụ rồi kết thúc lượt trả lời bằng câu soạn sẵn “*Đã hoàn tất truy vấn. Kết quả chính xác được hiển thị trong bảng phía trên.*” — trong khi không có bảng nào được gửi kèm. Người dùng nhận một lời khẳng định thành công cho một truy vấn chưa bao giờ chạy.

Cách kiểm chứng: dừng lại đúng mã nguồn cũ và gửi câu hỏi “Có bao nhiêu đơn vị trong bảng `units`?”. Luồng sự kiện trả về **không chứa sự kiện dataframe** nào, chỉ có câu kết soạn sẵn. Với mã nguồn đã sửa, cùng câu hỏi đó trả về `total_units = 33`.

5.2. Sai sót 2: tiêu chí thành công không phát hiện được lỗi trên

Kịch bản k6 của đợt trước coi một request là thành công khi thoả hai điều kiện: mã trạng thái HTTP 200 và luồng sự kiện kết thúc bằng `data: [DONE]`. Cả hai điều kiện đều được thoả trong tình huống ở Sai sót 1. Đó là lý do bản trước ghi nhận “tỉ lệ thành công 100%” cho những đợt đo mà tăng cơ sở dữ liệu hỏng hoàn toàn.

Bộ kiểm thử đã được siết lại: một request chỉ được tính thành công khi thân phản hồi chứa sự kiện `"type": "dataframe"`, tức có dữ liệu thật trả về. Tiêu chí lỏng cũ được giữ lại thành một chỉ số riêng (`ai_stream_rate`) để hai cách đếm đối chiếu được với nhau. Bộ kiểm thử cũng tách riêng độ trễ theo từng câu hỏi, nhằm phân biệt nhánh sinh SQL tắt định với nhánh thực sự gọi mô hình (xem Mục 2.3).

5.3. Sai sót 3: rò tiến trình suy luận giữa các lượt đo

Đổi `OLLAMA_NUM_PARALLEL` đòi hỏi khởi động lại máy chủ Ollama. Việc kết thúc tiến trình `ollama.exe` **không** kết thúc tiến trình con `llama-server.exe` đang giữ trọng số mô hình. Mỗi lần đổi cấu hình trong đợt trước để lại một tiến trình mờ côi chiếm 4–6 GB *commit charge*.

Vì bộ nhớ thường trú (working set) của các tiến trình này gần bằng không, chúng không lộ diện khi quan sát mức tiêu thụ RAM thông thường. Đến khi được phát hiện, ba tiến trình mờ côi — khởi tạo lúc 09:22, 09:33 và 09:38, đúng cửa sổ thử nghiệm cấu hình — giữ tổng cộng **14,9 GB**, đẩy máy 13,8 GB RAM tới mức commit 53,7/55,8 GB. Lượt đo lại đầu tiên thất bại ngay khi nạp mô hình với lỗi `failed to allocate CPU buffer of size 469762048`.

Hệ quả về phương pháp: các cấu hình trong đợt trước được đo **nối tiếp nhau trên một máy ngày càng cạn bộ nhớ**, cấu hình đo sau cùng chịu điều kiện xấu nhất. Ảnh hưởng đã được định lượng: cùng cấu hình NP=4 ở mức 50 người dùng, máy còn tiến trình mờ côi cho **4,24 req/giây**, máy đã dọn sạch cho **6,06 req/giây** — chênh lệch **43%** chỉ do bộ nhớ.

Quy trình đo đã sửa

Mỗi lượt đo nay (1) dọn tiến trình `llama-server` mờ côi sau khi dừng Ollama, (2) ghi lại mức bộ nhớ trống và commit charge trước khi bắt đầu, (3) nạp âm mô hình trước khi phát tải, và (4) chỉ tính thành công khi có dữ liệu thật trả về. Toàn bộ số liệu từ đây trở đi được thu thập bằng quy trình này.

6. Tối ưu hoá tầng ứng dụng và kết quả đo lại

6.1. Các thay đổi đã thực hiện

Bảng 6: Danh mục thay đổi mã nguồn nhằm gỡ nút thắt tăng ứng dụng

Tệp	Vấn đề	Cách khắc phục
postgres/ sql_runner.py	Mở kết nối mới cho mỗi truy vấn; psycopg2 đồng bộ chạy thẳng trong vòng lặp sự kiện	Dùng <code>ThreadedConnectionPool</code> (4–16 kết nối) và đẩy truy vấn sang <code>ThreadPoolExecutor</code> ; thêm <code>statement_timeout</code> 20 giây; loại bỏ kết nối lỗi khỏi pool
postgres/ sql_runner.py (<i>vá bổ sung</i>)	Cấu hình mỗi kết nối bằng cách gán thuộc tính lên đối tượng <code>connection</code> — kiểu C của psycopg2 không cho phép, làm mọi truy vấn ném lỗi (xem Mục 5)	Chuyển sang <code>connection_factory</code> : lớp con chạy phần cấu hình ngay trong <code>__init__</code> , đúng một lần lúc mở kết nối, không cần cờ đánh dấu
postgres/ sql_runner.py (<i>vá bổ sung</i>)	Phân loại kết quả bằng từ khoá đầu câu lệnh, trả sai với truy vấn <code>WITH ... SELECT</code> , <code>SHOW</code> , <code>EXPLAIN</code>	Dựa vào <code>cursor.description</code> để biết máy chủ có gửi result set hay không
ollama/llm.py	Vòng lặp đọc chuỗi token đồng bộ chiếm dụng vòng lặp sự kiện suốt thời gian sinh câu trả lời	<code>send_request</code> chuyển sang <code>asyncio.to_thread</code> ; <code>stream_request</code> đọc token trên luồng riêng và chuyển về vòng lặp sự kiện qua hàng đợi
main.py	Cửa sổ ngữ cảnh 8.192 token quá dư; độ dài sinh không giới hạn	<code>num_ctx</code> = 4096, <code>num_predict</code> = 512; tham số pool đưa ra biến môi trường

Toàn bộ thay đổi nằm ở **cách gọi** cơ sở dữ liệu và mô hình, không sửa logic nghiệp vụ, không sửa câu lệnh SQL và không đổi phần cứng.

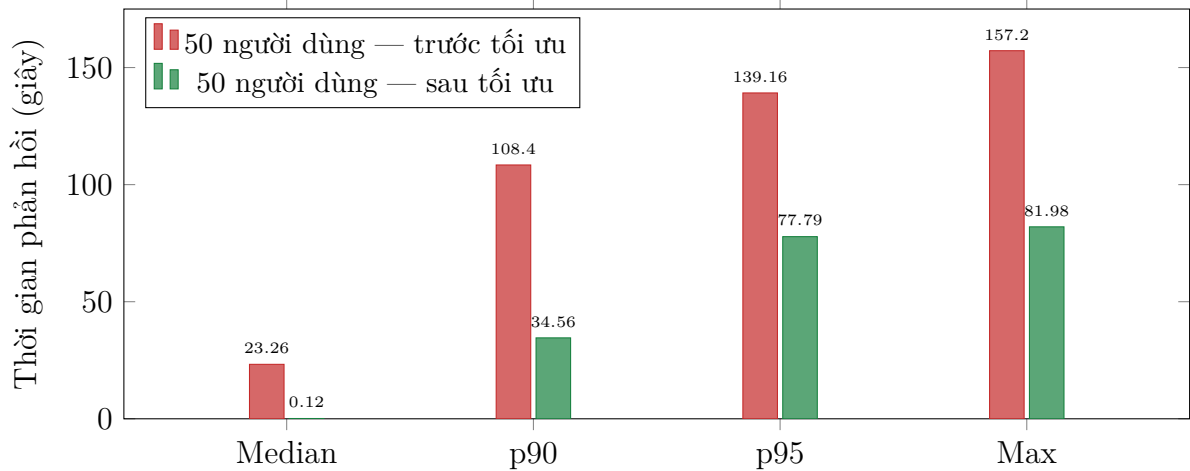
6.2. Kết quả đo lại ở mức 50 người dùng

Kịch bản B được chạy lại nguyên vẹn (cùng bộ câu hỏi, cùng cấu hình `ramping-vus`, cùng máy), với `OLLAMA_NUM_PARALLEL` vẫn để mặc định bằng 1 — tức chỉ đo tác dụng của phần sửa mã nguồn. Kết quả trình bày ở Bảng 7.

Bảng 7: So sánh trước và sau tối ưu tại mức 50 người dùng đồng thời

Chỉ số	Trước tối ưu	Sau tối ưu	Thay đổi
Tổng số request	132	842	+538%
Thông lượng (req/giây)	0,70	4,43	+533%
Tỉ lệ thành công	87,1%	100%	+12,9 điểm
Số lỗi	17	0	-100%
Nhanh nhất (min), giây	1,34	0,05	-96%
Trung vị (median), giây	23,26	0,12	-99,5%
Trung bình (avg), giây	36,28	7,58	-79%
p90, giây	108,40	34,56	-68%
p95, giây	139,16	77,79	-44%
Chậm nhất (max), giây	157,20	81,98	-48%
1 người dùng (câu chạm CSDL)	≈ 4 giây	0,105 giây	-97%
1 người dùng (câu gọi LLM)	—	1,99 giây	—

Cột “trước tối ưu” lấy từ đợt đo hiện trạng, khi tầng truy vấn còn hoạt động bình thường; tỉ lệ thành công của cột này được đếm bằng tiêu chí lỏng cũ (xem Mục 5), trong khi cột “sau tối ưu” dùng tiêu chí chặt buộc phải có dữ liệu trả về.



Hình 3: Phân bố thời gian phản hồi tại mức 50 người dùng, trước và sau tối ưu. Trung vị gần như biến mất khỏi biểu đồ (0,12 giây) trong khi đuôi trễ vẫn cao: phân bố đã tách hẳn thành hai cụm.

6.3. Diễn giải kết quả

Sau khi sửa, phân bố độ trễ **không còn là một cụm** mà tách hẳn thành hai, tương ứng với hai nhánh xử lý đã mô tả ở Mục 2.3. Bảng 8 tách số đo theo từng câu hỏi.

Bảng 8: Độ trễ tách theo câu hỏi, mức 50 người dùng, sau tối ưu mã nguồn (NP=1)

Câu hỏi	Nhánh	Số lần	Trung vị	p95
Có bao nhiêu người dùng trong hệ thống?	SQL tất định	126	0,10	0,32
Có bao nhiêu đơn vị trong bảng units?	SQL tất định	165	0,11	0,37
Có bao nhiêu spdv_tickets?	SQL tất định	150	0,12	0,34
Đếm số người dùng theo trạng thái	SQL tất định	134	0,10	0,43
Liệt kê 5 đơn vị đầu tiên theo tên	Gọi LLM	108	69,74	80,72
Có bao nhiêu sự kiện trong spdv_ticket_events?	SQL tất định	159	0,11	0,30

Đơn vị thời gian: giây. Cả sáu câu đều đạt 100% có dữ liệu trả về.

Ba nhận định rút ra từ bảng này:

- **Toàn bộ độ trễ đuôi đến từ một câu hỏi duy nhất.** Năm câu đi nhánh SQL tất định giữ p95 dưới 0,5 giây ngay ở mức 50 người dùng đồng thời — tức tăng ứng dụng và tầng cơ sở dữ liệu **không hề là nút thắt**. Chỉ số p95 tổng thể 77,79 giây thực chất là p95 của riêng nhánh gọi mô hình, pha loãng qua tỉ lệ 1/6.
- **Hệ số xếp hàng của nhánh LLM là 35 lần.** Cùng câu hỏi đó, khi máy rảnh chỉ mất 1,99 giây (đo 5 lần ở cấu hình cuối cùng, dao động 1,86–2,09); dưới 50 người dùng trung vị lên 69,74 giây. Chênh lệch này là thời gian *chờ hàng đợi*, không phải thời gian tính toán.
- **Trung vị tổng thể không còn là chỉ số có ý nghĩa.** Với phân bố hai cụm, trung vị chỉ cho biết cụm nào đông hơn (5/6 số request đi nhánh nhanh), chứ không mô tả trải nghiệm của người dùng hỏi câu cần suy luận.

Đánh giá đợt tối ưu

Đợt tối ưu đạt mục tiêu về **độ tin cậy** (lỗi 13% → 0%) và **thông lượng** (+533%), nhưng chưa đạt mục tiêu về **độ trễ** trên nhánh gọi mô hình: p95 của nhánh này vẫn ở mức hàng chục giây, cách rất xa ngưỡng SLA 10 giây. Phần còn lại phụ thuộc vào năng lực suy luận, tức phần cứng hoặc lựa chọn mô hình.

7. Thử nghiệm cấu hình tầng suy luận

Sau đợt tối ưu mã nguồn, các phương án cấu hình tầng suy luận được thử nghiệm để xác định xem có thể gỡ tiếp nút thắt mà không cần đầu tư phần cứng hay không. Mọi thử nghiệm dùng chung kịch bản, bộ câu hỏi và mã nguồn đã tối ưu; chỉ khác cấu hình Ollama hoặc mô hình.

Ba giá trị NUM_PARALLEL đã được **đo lại đầy đủ** ở cả hai mức tải bằng quy trình đã sửa ở Mục 5 — sáu lượt đo, mỗi lượt khởi động lại máy chủ mô hình và dọn sạch tiến trình cũ. Hai phương án còn lại (GPU tích hợp và mô hình 1B) chưa được đo lại; trạng thái của chúng ghi ở Mục 11.

7.1. Số lượng luồng suy luận song song

Rà soát nhật ký Ollama phát hiện tham số mặc định trên máy kiểm thử là `OLLAMA_NUM_PARALLEL: 1` — nghĩa là máy chủ mô hình chỉ xử lý **một** yêu cầu tại một thời điểm, mọi yêu cầu còn lại xếp hàng. Đây là một tham số cấu hình, không phải giới hạn phần cứng.

Bảng 9: So sánh các giá trị `NUM_PARALLEL`, đo lại trên mã nguồn đã sửa

Cấu hình	Request	RPS	Trung vị	p90	p95	Max
<i>Mức 30 người dùng</i>						
NP=1 (CPU)	879	3,64	0,11	16,79	33,01	51,57
NP=4 (CPU)	1.077	4,61	0,16	15,40	27,26	61,85
NP=8 (CPU)	655	3,03	2,64	15,32	19,69	116,97
<i>Mức 50 người dùng</i>						
NP=1 (CPU)	842	4,43	0,12	34,56	77,79	81,98
NP=4 (CPU)	1.140	6,06	0,11	44,67	46,34	48,54
NP=8 (CPU)	696	4,19	9,40	21,98	27,85	42,34

Đơn vị thời gian: giây. NP = `OLLAMA_NUM_PARALLEL`. Cả sáu lượt đo đều đạt tỉ lệ thành công 100% theo tiêu chí chặt (có dữ liệu trả về); tổng cộng 5.289 request, không có lượt nào trả lời rỗng.

Nâng `NUM_PARALLEL` từ 1 lên 4 làm tăng thông lượng **27%** ở mức 30 người dùng và **37%** ở mức 50 người dùng. Cơ chế ở đây là gộp lô liên tục (continuous batching): sinh token bị chặn bởi băng thông bộ nhớ chứ không phải bởi phép tính, nên xử lý bốn luồng cùng lúc chia sẻ được chi phí đọc trọng số mô hình. Đây là lý do batching vẫn có hiệu quả ngay cả khi chạy hoàn toàn trên CPU.

7.2. Cảnh báo khi đọc bảng trên: p95 của NP=8 là bẫy

Ở mức 50 người dùng, NP=8 có p95 (27,85 giây) *tốt hơn* NP=4 (46,34 giây). Đây **không** phải một cải thiện, và chọn cấu hình theo cột p95 sẽ dẫn tới kết luận sai.

Bảng chứng nằm ở nhánh không liên quan gì tới mô hình. Với NP=8, năm câu hỏi đi nhánh SQL tất định — vốn chỉ chạm Postgres — **cũng chậm đi hàng chục lần**: trung vị 0,11 → 4,75–9,40 giây, p95 0,4 → 20–22 giây. Không có cơ chế nào ở tầng suy luận giải thích được điều này. Nguyên nhân là bộ nhớ: mô hình chiếm 5.658 MB, đầy bộ nhớ trống của máy xuống 0,8 GB và toàn hệ thống rơi vào trạng thái tráo tràng liên tục.

Khi mọi request đều chậm, mỗi người dùng ảo quay vòng chậm hơn, nên số request gọi mô hình dồn vào hàng đợi cũng ít đi — p95 của nhánh LLM vì thế *ngắn lại*. Chỉ số phản ánh đúng thực tế ở đây là thông lượng: 6,06 → 4,19 req/giây, tức **mất 31%** năng lực.

7.3. Các phương án bị bác bỏ

- **Tăng `NUM_PARALLEL` lên 8** — *đã đo lại, bác bỏ*. Bộ nhớ mô hình phình từ 3.864 MB lên 5.658 MB. Thông lượng giảm 34% ở mức 30 người dùng và 31% ở mức 50 người dùng. Cơ chế là **áp lực bộ nhớ**, không phải tranh chấp nhân CPU như bản báo cáo

trước nhận định — xem lập luận ở mục trên. Giá trị 4 là điểm tối ưu trên phần cứng này.

- **Dùng GPU tích hợp (Radeon 780M)** — *chưa đo lại*. Đợt trước ghi nhận thông lượng giảm 47% so với CPU, với giả thích rằng iGPU dùng chung băng thông bộ nhớ hệ thống với CPU. Kết luận này thu được bằng quy trình đo có ba sai sót ở Mục 5 nên **chưa đủ căn cứ**; cần đo lại trước khi dùng để ra quyết định.
- **Đổi sang mô hình nhỏ hơn (llama3.2:1b)** — *chưa đo lại*. Đợt trước ghi nhận mỗi lượt gọi mô hình mất 23–27 giây thay vì 2–4 giây của bản 3B, do mô hình 1B tuân thủ chỉ thị kém nên sinh dài dòng và lặp vòng gọi công cụ. Quan sát này hợp lý về mặt cơ chế nhưng cũng thu được bằng quy trình đo cũ, nên cần đo lại để khẳng định.

7.4. Kết quả với cấu hình tốt nhất

Bảng 10: Kết quả cuối cùng: mã nguồn đã tối ưu và NUM_PARALLEL=4

Chỉ số	30 user (ban đầu)	30 user (cuối cùng)	50 user (ban đầu)	50 user (cuối cùng)
Tổng số request	177	1.077	132	1.140
Thông lượng (req/giây)	0,75	4,61	0,70	6,06
Tỉ lệ thành công	95,0%	100%	87,1%	100%
Số lỗi	9	0	17	0
Trung vị, giây	19,43	0,16	23,26	0,11
p90, giây	32,71	15,40	108,40	44,67
p95, giây	55,11	27,26	139,16	46,34
Max, giây	89,28	61,85	157,20	48,54

So với hiện trạng ban đầu, thông lượng tăng **515%** ở mức 30 người dùng và **766%** ở mức 50 người dùng, tỉ lệ lỗi về 0% ở cả hai mức — toàn bộ đạt được bằng sửa mã nguồn và đổi một tham số cấu hình, **không tốn chi phí phần cứng**.

Cần đọc con số này đúng phạm vi: phần lớn mức tăng thông lượng đến từ nhánh SQL tắt định, nơi độ trễ giảm từ vài giây xuống 0,105 giây nhờ dùng lại kết nối và bỏ chặn vòng lặp sự kiện. Nhánh gọi mô hình cải thiện ít hơn nhiều và vẫn là toàn bộ phần đuôi trễ.

7.5. Vì sao vẫn không đạt p95 dưới 10 giây

Với phân bố hai cụm đã chỉ ra ở Bảng 8, câu hỏi này quy về một bài toán hàng đợi trên **riêng nhánh gọi mô hình**. Ba đại lượng đều đo được trực tiếp:

- **Thời gian phục vụ khi máy rảnh**: 1,99 giây cho một lượt hỏi cần suy luận (đo 5 lần liên tiếp, dao động 1,86–2,09).
- **Năng lực bão hoà của nhánh này**: ở mức 50 người dùng với NP=4, 186 lượt gọi mô hình hoàn tất trong 188 giây, tức $\mu \approx 0,99$ req/giây.
- **Tỉ lệ request cần suy luận**: 1/6.

Hai con số đầu đã giải thích trọn vẹn trần thông lượng của cả hệ thống: $6 \times 0,99 = 5,94$ req/giây, khớp với 6,06 req/giây đo được. Nói cách khác, **toàn bộ năng lực hệ thống bị quyết định bởi một nhánh chiếm 1/6 lưu lượng**.

Để độ trễ nhánh này giữ dưới $W = 10$ giây, hàng đợi phải chưa bão hoà. Với mô hình M/M/1, $W = 1/(\mu - \lambda)$ cho $\lambda \approx 0,9$ req/giây. Mỗi người dùng ảo hỏi liên tục không nghỉ, nên chu kỳ trung bình của một người là $\frac{5}{6}(0,105) + \frac{1}{6}W$ giây. Cân bằng hai vế:

$$0,9 = \frac{N/6}{0,0875 + 10/6} \implies N \approx 9,5$$

Tức khoảng **9–10 người dùng đồng thời** — và đây là ước lượng cho độ trễ *trung bình*; ràng buộc theo p95 sẽ chặt hơn. Muốn phục vụ 50 người dùng trong ngưỡng 10 giây cần $\mu \approx 5$ req/giây trên nhánh suy luận, tức **gấp 5 lần** năng lực hiện có. Khoảng cách này không lấp được bằng cấu hình hay mã nguồn: giá trị NUM_PARALLEL tốt nhất đã được xác định bằng thực nghiệm, và phần còn lại là tốc độ sinh token trên CPU.

Sức chứa với ngưỡng p95 dưới 10 giây

Từ mô hình hàng đợi trên, cấu hình cuối cùng đáp ứng ngưỡng 10 giây ở khoảng **8–10 người dùng đồng thời**. Cần nhấn mạnh rằng ràng buộc này chỉ áp lên 1/6 lưu lượng: năm trong sáu câu hỏi giữ p95 dưới nửa giây ở mọi mức tải đã đo. Muốn phục vụ 30–50 người dùng *với câu hỏi cần suy luận* trong ngưỡng này thì bắt buộc phải đầu tư GPU rồi hoặc chuyển sang API LLM đám mây.

8. Đánh giá theo tiêu chí nghiệm thu (SLA)

Bảng 11 đối chiếu kết quả đo với các ngưỡng chất lượng dịch vụ đề xuất cho một hệ thống hỏi đáp tương tác. Với cấu hình cuối cùng, hai tiêu chí về **độ tin cậy đã đạt** ở cả mức 30 và 50 người dùng; tiêu chí về **độ trễ vẫn không đạt** do giới hạn tốc độ suy luận trên CPU (xem Mục 7.5).

Bảng 11: Đối chiếu kết quả với tiêu chí nghiệm thu đề xuất

Tiêu chí	Ngưỡng mục tiêu	30 user (ban đầu)	30 user (cuối cùng)	50 user (cuối cùng)
Độ trễ p95 (toàn bộ)	< 10 giây	Không đạt (55,11 giây)	Không đạt (27,26 giây)	Không đạt (46,34 giây)
— nhánh SQL tất định	< 10 giây	—	Đạt (0,52 giây)	Đạt (0,37 giây)
— nhánh gọi LLM	< 10 giây	—	Không đạt (29,64 giây)	Không đạt (47,79 giây)
Tỉ lệ thành công	$\geq 99\%$	Không đạt	Đạt (100%)	Đạt (100%)
Tỉ lệ lỗi	$\leq 1\%$	Không đạt	Đạt (0%)	Đạt (0%)

Tỉ lệ thành công của hai cột “cuối cùng” được đếm bằng tiêu chí chặt (bắt buộc có dữ liệu trả về); cột “ban đầu” dùng tiêu chí lỏng cũ.

Tách theo nhánh cho thấy tiêu chí độ trễ **đã đạt cho 5/6 lưu lượng** và chỉ không đạt trên nhánh gọi mô hình. Đây là thông tin có giá trị khi cân nhắc phạm vi triển khai: nếu tập câu hỏi thực tế nghiêng về nhóm đếm và tổng hợp quen thuộc, sức chứa cao hơn nhiều so với con số tổng gộp.

Sức chứa an toàn ước tính

Nếu tiêu chí bắt buộc là **không mất request**, cấu hình cuối cùng đã phục vụ được **50 người dùng đồng thời** với tỉ lệ thành công 100% (5.289 request qua sáu lượt đo, không có lượt nào trả lời rỗng). Nếu tiêu chí bao gồm cả **p95 dưới 10 giây cho câu hỏi cần suy luận**, sức chứa chỉ ở mức **8–10 người dùng đồng thời**, và muốn nâng lên thì bắt buộc phải đầu tư vào tăng suy luận theo các khuyến nghị ở Mục 11.

9. Giới hạn của kiểm thử

Kết quả trong báo cáo cần được diễn giải trong phạm vi các giới hạn sau:

- **Mô hình một máy:** ứng dụng, mô hình Ollama và công cụ k6 chạy trên cùng một máy, nên có sự cạnh tranh tài nguyên CPU giữa bên tạo tải và bên xử lý.
- **Phần cứng tiêu dùng:** máy dùng GPU tích hợp, không đại diện cho cấu hình máy chủ production có GPU rời; kết quả tuyệt đối sẽ khác trên hạ tầng mạnh hơn.
- **Câu hỏi đơn giản:** bộ câu hỏi thiên về đếm/tổng hợp; câu hỏi phức tạp (nhiều join, cửa sổ thời gian) sẽ làm tăng cả chi phí SQL lẫn suy luận.
- **Độ dài kiểm thử:** mỗi kịch bản kéo dài vài phút, chưa đánh giá độ ổn định dài hạn (rò rỉ bộ nhớ, suy giảm hiệu năng theo thời gian).
- **Cơ sở dữ liệu đám mây:** độ trễ mạng tới Neon có thể dao động theo thời điểm, ảnh hưởng nhỏ đến kết quả.
- **Bộ câu hỏi lặp lại và phần lớn không gọi mô hình:** sáu câu hỏi quay vòng ngẫu nhiên, trong đó chỉ một câu đi qua tầng suy luận. Vì vậy các chỉ số tổng gộp (RPS, trung vị, p95) **không đại diện** cho một tải thực tế có tỉ lệ câu hỏi cần suy luận cao hơn. Khi tỉ lệ này tăng, thông lượng sẽ giảm gần như tuyến tính — trần hệ thống là μ/f với $\mu \approx 1$ req/giây và f là tỉ lệ câu cần suy luận.
- **Hai phương án chưa được đo lại:** kết luận về GPU tích hợp và về mô hình llama3.2:1b vẫn dựa trên số liệu thu bằng quy trình đo cũ (Mục 5), nên chưa đủ căn cứ.
- **Áp lực bộ nhớ của chính máy đo:** máy có 13,8 GB RAM khả dụng nhưng các ứng dụng nền chiếm sẵn khoảng 39 GB commit charge. Cấu hình NUM_PARALLEL=8 đẩy bộ nhớ trống xuống 0,8 GB. Kết quả của cấu hình này vì vậy phản ánh giới hạn bộ nhớ của máy đo nhiều hơn là đặc tính của tham số.

10. Kết luận

Sau tối ưu, hệ thống phục vụ được 50 người dùng đồng thời không mất request, nhưng độ trễ vẫn bị chặn bởi phần cứng.

Tối ưu mã nguồn cộng với việc đặt `OLLAMA_NUM_PARALLEL=4` đưa tỉ lệ lỗi từ 13% về 0%, nâng thông lượng từ 0,70 lên **6,06 request/giây** (+766%) và giảm p95 từ 139 giây xuống **46,3 giây** (-67%) — toàn bộ trên cùng phần cứng, không phát sinh chi phí. Tuy nhiên p95 của nhánh gọi mô hình vẫn vượt xa ngưỡng SLA 10 giây.

Bài học rút ra: kết luận ban đầu quy toàn bộ nút thắt cho mô hình ngôn ngữ là chưa đầy đủ. Một phần đáng kể độ trễ và **toàn bộ lỗi** thực ra đến từ tầng ứng dụng — cụ thể là việc gọi thư viện đồng bộ trong vòng lặp sự kiện bất đồng bộ và việc không dùng connection pool. Đây là những lỗi sửa được bằng mã nguồn với chi phí bằng không, và nên được rà soát trước khi kết luận về nhu cầu phần cứng.

Sau khi lớp này được gỡ bỏ, nút thắt còn lại là năng lực suy luận thật sự, và nó **nằm gọn trong một nhánh chiếm 1/6 lưu lượng**. Năm trong sáu câu hỏi được trả lời dưới nửa giây ngay ở mức 50 người dùng; toàn bộ phần đuôi trễ đến từ câu còn lại. Một phần của nút thắt này cũng chỉ là cấu hình: `OLLAMA_NUM_PARALLEL` để mặc định bằng 1 khiến máy chủ mô hình xử lý tuần tự, nâng lên 4 làm thông lượng tăng 37%; nâng tiếp lên 8 thì *giảm* 31% do áp lực bộ nhớ (Mục 7).

Vì vậy khoảng cách còn lại tới ngưỡng p95 dưới 10 giây (cần gấp ≈ 5 lần năng lực suy luận ở mức 50 người dùng) **không thể lấp bằng cấu hình hay mã nguồn**. Để phục vụ nhiều người dùng hơn với độ trễ chấp nhận được, bắt buộc phải đầu tư vào tầng suy luận theo các khuyến nghị dưới đây.

Bài học về phương pháp kiểm thử

Bài học đáng giá nhất của đợt này không nằm ở con số hiệu năng mà ở quy trình đo. Một bộ kiểm thử tải chỉ kiểm tra mã trạng thái và dấu kết thúc luồng đã báo “100% thành công” trong suốt một đợt đo mà **tầng cơ sở dữ liệu hỏng hoàn toàn** — vì ứng dụng vẫn trả về HTTP 200 kèm một câu khẳng định soạn sẵn. Nguyên tắc rút ra: **tiêu chí thành công của kiểm thử tải phải kiểm tra nội dung trả về**, không chỉ hình thức giao thức. Kèm theo đó là hai thói quen bắt buộc: dọn sạch tiến trình con của máy chủ mô hình giữa các lượt đo, và ghi lại trạng thái bộ nhớ của máy trước mỗi lượt.

11. Khuyến nghị cải thiện

11.1. Đã thực hiện trong đợt tối ưu này

Bảng 12: Các hạng mục đã hoàn thành và kết quả thực đo

Trạng thái	Giải pháp	Kết quả thực đo
Hoàn thành	Đưa lời gọi <code>psycopg2</code> và <code>ollama</code> ra khỏi vòng lặp sự kiện (<code>thread pool</code> và hàng đợi)	Tỉ lệ lỗi 13% → 0%; thông lượng +533%
Hoàn thành	Connection pool tới Neon (4–16 kết nối) thay cho mở kết nối mỗi truy vấn	Truy vấn chậm CSDL: $\approx$ 4 giây → 0,105 giây
Hoàn thành	Sửa lỗi làm hỏng toàn bộ tầng truy vấn: dùng <code>connection_factory</code> thay cho gán thuộc tính lên đối tượng kết nối	Khôi phục 100% tỉ lệ trả về dữ liệu thật (Mục 5)
Hoàn thành	Nhận biết <code>result set</code> bằng <code>cursor.description</code> thay cho từ khoá đầu câu lệnh	Truy vấn <code>WITH ... SELECT</code> trả đúng dữ liệu thay vì số dòng
Hoàn thành	Giới hạn <code>statement_timeout</code> 20 giây và <code>num_predict</code> 512 token	Chặn đuôi trễ: max 157 → 48,5 giây
Hoàn thành	Đặt <code>OLLAMA_NUM_PARALLEL=4</code> thay cho mặc định 1	Thông lượng +37% ở mức 50 người dùng
Hoàn thành	Siết tiêu chí thành công của k6: bắt buộc có sự kiện <code>dataframe</code> ; tách độ trễ theo từng câu hỏi	Phát hiện được lỗi trả lời rỗng mà tiêu chí cũ bỏ sót

11.2. Đã thử: một bị bác bỏ, hai cần đo lại

Ba phương án dưới đây từng nằm trong danh mục khuyến nghị. Một phương án đã được đo lại bằng quy trình đã sửa và bị bác bỏ dứt khoát; hai phương án còn lại **cần đo lại** vì kết luận cũ dựa trên số liệu không đủ tin cậy (chi tiết ở Mục 5 và Mục 7).

Bảng 13: Các phương án đã thử nghiệm và trạng thái hiện tại

Kết luận	Phương án	Kết quả đo
Loại	Nâng <code>OLLAMA_NUM_PARALLEL</code> lên 8	Thông lượng −31% ở mức 50 người dùng; nguyên nhân là áp lực bộ nhớ, 4 là điểm tối ưu
Cần đo lại	Bật tăng tốc GPU tích hợp Radeon 780M (<code>OLLAMA_IGPU_ENABLE</code>)	Số cũ: thông lượng −47% — thu bằng quy trình đo có sai sót
Cần đo lại	Đổi sang mô hình nhỏ hơn <code>llama3.2:1b</code>	Số cũ: mỗi lượt gọi 23–27 giây — thu bằng quy trình đo có sai sót

11.3. Còn lại — cần đầu tư

Bảng 14: Các khuyến nghị nâng cao khả năng chịu tải

Ưu tiên	Giải pháp	Kỳ vọng
Cao	Trang bị GPU rời cho máy chủ suy luận	Tốc độ sinh token nhanh 10–20 lần — đòn bẩy duy nhất đủ lớn để đạt p95 dưới 10 giây
Cao	Hoặc chuyển sang API LLM đám mây (Anthropic/OpenAI) cho tải cao	Bỏ hoàn toàn trần CPU cục bộ; đổi lại phát sinh chi phí theo token
Trung bình	Cache câu hỏi và câu lệnh SQL lặp lại	Giảm số lần gọi mô hình. Lưu ý: bộ câu hỏi kiểm thử chỉ có 6 câu lặp lại nên cache sẽ cho tỉ lệ trùng gần 100% và kết quả đo sẽ không phản ánh tải thật; cần đánh giá lại bằng bộ câu hỏi đa dạng hơn
Trung bình	Chạy Uvicorn nhiều worker (<code>--workers 4</code>) hoặc đặt sau Gunicorn	Tận dụng đa nhân cho tầng ứng dụng (lợi ích hạn chế khi CPU đã bị suy luận chiếm)
Thấp	Thêm hàng đợi và thông báo “đang xử lý” cho người dùng	Cải thiện trải nghiệm khi tải cao

Người thực hiện
(Đã lập báo cáo)

Lê Đức Anh

Phụ lục A. Kịch bản k6 (rút gọn)

Listing 2: Logic phát tải và đo lường của mỗi người dùng ảo

```
import http from 'k6/http';
import { check } from 'k6';
import { Trend, Rate, Counter } from 'k6/metrics';

const aiResponseTime = new Trend('ai_response_time', true);
const aiSuccessRate = new Rate('ai_success_rate'); // chat: phải có dữ liệu
const aiStreamRate = new Rate('ai_stream_rate'); // long: tiêu chí cũ
const aiDataRate = new Rate('ai_data_rate');
const aiErrors = new Counter('ai_errors');
const perQuestion = QUESTIONS.map((_, i) => new Trend(`ai_time_q${i+1}`, true));

export default function () {
  // conversation_id duy nhất -> tránh khoa 409 theo hỏi thoại
  const index = Math.floor(Math.random() * QUESTIONS.length);
  const uid = `lt-${__VU}-${__ITER}-${Date.now()}-${Math.random()}`;

  const res = http.post(ENDPOINT,
    JSON.stringify({ message: QUESTIONS[index], conversation_id: uid, request_id: uid }),
    { headers: { 'Content-Type': 'application/json' }, timeout: '180s' });

  const body = String(res.body);
  const streamed = res.status === 200 && body.includes('data: [DONE]');
  // Điểm mau chốt: stream tron ven KHONG dong nghĩa với có dữ liệu trả về.
  const hasData = body.includes('"type":"dataframe"');
  const ok = streamed && hasData;

  aiResponseTime.add(res.timings.duration);
  perQuestion[index].add(res.timings.duration);
  aiStreamRate.add(streamed);
  aiDataRate.add(hasData);
  aiSuccessRate.add(ok);
  if (!ok) aiErrors.add(1);

  check(res, {
    'HTTP 200': (r) => r.status === 200,
    'stream hoàn tất': () => streamed,
    'có dữ liệu trả về': () => hasData,
  });
}
```

Mã nguồn đầy đủ và số liệu thô có trong thư mục loadtest/ của kho mã. Phần dùng chung của hai kịch bản nằm ở chat_probe.js; kịch bản ai_load_test.js (30 user) và ai_load_test_50.js (50 user).

Số liệu **hiện trạng ban đầu**: k6_summary.json, k6_summary_50.json. Số liệu **đo lại** trên mã nguồn đã sửa, đặt tên theo mẫu k6_{mức tải}_np{NP}_v2.json — sáu tệp cho ba giá trị NUM_PARALLEL ở hai mức tải.

Các tệp k6_summary_50_optimized.json, k6_30_np4.json, k6_50_np4.json, k6_50_np8.json

và `k6_50_gpu.json` là số liệu của đợt đo cũ; giữ lại để đối chiếu nhưng **không dùng cho kết luận nào** trong bản báo cáo này (xem Mục [5](#)).

Các thay đổi mã nguồn của đợt tối ưu nằm ở: `src/vanna/integrations/postgres/sql_runner.py`, `src/vanna/integrations/ollama/llm.py` và `main.py`.

Tài liệu tham khảo

- [1] Grafana Labs, *k6 — Load testing tool for engineering teams*, tài liệu công cụ kiểm thử tải mã nguồn mở, phiên bản 2.1.0. <https://k6.io/docs/>. Truy cập ngày 04/08/2026.
- [2] Vanna AI, *Vanna 2.0: Turn Questions into Data Insights*, GitHub repository [vanna-ai/vanna](https://github.com/vanna-ai/vanna), phiên bản mã nguồn tham chiếu 2.0.2. <https://github.com/vanna-ai/vanna>. Truy cập ngày 04/08/2026.
- [3] Ollama, *Ollama — Run large language models locally*, tài liệu cấu hình song song (OLLAMA_NUM_PARALLEL) và phục vụ mô hình. <https://github.com/ollama/ollama>. Truy cập ngày 04/08/2026.
- [4] Neon, *Connection pooling*, tài liệu kết nối PostgreSQL và pooled connection cho ứng dụng. <https://neon.com/docs/connect/connection-pooling>. Truy cập ngày 04/08/2026.